Nombre: JOSE BONILLA LARA **Nombre:** IÑAKI ALEXANDER GONZALEZ ESTRADA **Nombre:** ALEJANDRÍA QUEVEDO VELEZ	**Matrícula**: 2144787 **Matricula:** 2144262 **Matricula:** 2144956
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad:	**Actividad**: Caso de estudio equipo 9
Fecha: 04/09/2026	

Bibliografia (Formato APA):

Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional.

Objetivo y alcance

Este documento cubre las etapas 1 y 2 de la actividad: prepara el caso (casos de uso del proceso principal) y elige el patrón de lógica de negocio (Transaction Script, Table Module o Domain Model) que mejor responde a la complejidad real de las reglas del Hotel Costa Azul, justificándolo con evidencia.

El criterio no es elegir el patrón más moderno ni el que genere más clases, sino el que se ajuste a la complejidad de las reglas que el hotel necesita proteger. La evidencia sale únicamente de los productos de la Unidad 2: la entrevista de levantamiento, el BRD, las 15 historias de usuario (HU01 a HU15), los criterios de aceptación, el BPMN y el glosario.

El proceso principal elegido para el diseño detallado es el ciclo central del hotel: reserva, check-in, estancia, check-out y limpieza. Se usan 5 casos de uso, cada uno con un flujo normal y un flujo donde la operación puede rechazarse.

Casos de uso seleccionados

Se eligieron cinco casos de uso que cubren el ciclo central del hotel y los puntos donde una operación puede rechazarse. Las reglas se identifican con el código R1 a R12 del documento 02 (mapa de dominio).

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Confirmar reserva	Reservaciones (a solicitud del huésped)	Límite fijo de 120 habitaciones y disponibilidad real (R1); tarifa según temporada, tipo de habitación y condiciones comerciales (R2); rechazo si no hay disponibilidad.	Reserva confirmada con tarifa calculada y disponibilidad descontada, o rechazo sin reserva.
CU-02	Realizar check-in	Recepción	Identidad (INE o pasaporte) y garantía (tarjeta o depósito en efectivo) obligatorias (R3); solo se asigna una habitación Disponible (R4); la llegada anticipada depende del estado de la habitación (HU12).	Estancia iniciada, habitación Ocupada y Cuenta abierta.
CU-03	Registrar consumo adicional	Recepción, restaurante o lavandería	Solo recepción, restaurante y lavandería cargan a la cuenta, y la autorización se valida en el instante (R7); todos los cargos van a una sola cuenta (R8).	Consumo cargado a la Cuenta con origen y usuario.
CU-04	Cambiar de habitación	Recepción (por confirmar con HU11)	La habitación destino debe estar Disponible y no Fuera de Servicio (R4); el cambio guarda motivo, hora, usuario, habitación anterior y nueva (R9).	Estancia con la nueva habitación, cambio registrado y las dos habitaciones actualizadas.
CU-05	Hacer check-out y liberar para limpieza	Recepción y limpieza	Cargos consolidados y saldo liquidado (R8); la habitación liberada pasa por limpieza antes de volver a estar disponible y recepción debe verlo de inmediato (R5).	Cuenta liquidada, Estancia cerrada y Habitación liberada que vuelve a Disponible cuando limpieza la marca limpia.

Verificación contra la Unidad 2. Cada caso aparece en las historias de usuario indicadas y en el BPMN, por lo que no hay correcciones de alcance. La tabla muestra el flujo normal y el flujo con excepción o rechazo de cada uno.

ID	Historias / BPMN	Flujo normal	Flujo con excepción o rechazo

ID	Historias / BPMN	Flujo normal	Flujo con excepción o rechazo
CU-01	HU01, HU02	El huésped indica fechas, número de huéspedes y tipo. El sistema muestra la disponibilidad real sobre el límite de 120 habitaciones, calcula la tarifa según temporada, tipo y condiciones comerciales, y se confirma la reserva.	No hay habitación disponible: el flujo normal no continúa. La acción posterior está pendiente de validar.
CU-02	HU03, HU12	Recepción verifica identidad (INE o pasaporte) y garantía (tarjeta de crédito o depósito en efectivo) y asigna habitación.	La verificación de identidad o garantía falla: el caso no define qué sucede. En llegada anticipada, si la habitación no está lista, el huésped espera.
CU-03	HU04	Recepción, restaurante o lavandería cargan un consumo a la cuenta de la estancia.	El rol que intenta el cargo no está autorizado. Hoy no existe un candado que lo valide en el instante.
CU-04	HU11, HU10	Se registra el cambio con motivo, hora, usuario y habitación anterior y nueva.	La habitación destino no está disponible o está fuera de servicio, y no puede asignarse.
CU-05	HU05, HU06	Se revisan y consolidan los cargos, se liquida el saldo y se libera la habitación. Limpieza la marca como limpia y recepción la ve disponible.	La cuenta tiene cargos sin consolidar o saldo sin liquidar: la estancia no debería cerrarse (regla derivada del glosario, por validar).

Las demás historias (HU07 cancelaciones y no-shows, HU08 salida anticipada, HU09 extensión, HU13 a HU15 reportes, tarifas y auditoría) se usan como evidencia complementaria en la sección de decisión.

Patrones evaluados

Los tres patrones se diferencian en dónde viven las reglas de negocio y qué tan bien soportan que esas reglas crezcan. Aquí solo se describe qué es cada uno.

- **Transaction Script:** organiza la lógica como un procedimiento por operación (por ejemplo, un script para confirmar reserva y otro para hacer check-out).
- **Table Module:** una sola clase por concepto de datos (por ejemplo, un módulo de habitaciones) que procesa todos sus registros.
- **Domain Model:** objetos de negocio con datos y comportamiento (Reserva, Estancia, Habitación, Cuenta) que protegen sus propias reglas y estados.

Ventajas, riesgos y consecuencias de cada alternativa

La tabla evalúa cada patrón en el contexto del Hotel Costa Azul: qué ventaja real ofrece, qué riesgo trae para este caso y qué consecuencia tendría elegirlo.

Patrón	Ventajas en el hotel	Riesgos en el hotel	Consecuencias de elegirlo	Resultado

Patrón	Ventajas en el hotel	Riesgos en el hotel	Consecuencias de elegirlo	Resultado
Transaction Script	Es el más rápido de construir. Las operaciones del hotel (reservar, hacer check-in, hacer check-out) son procedimientos claros de principio a fin. Sería correcto si las reglas fueran pocas y no se cruzaran.	La disponibilidad y la tarifa se usan en reserva, extensión y salida anticipada, así que se repetirían en cada script. Notificar y registrar quedarían como pasos manuales dentro del script, que es la falla actual del hotel.	Cada regla pendiente (penalizaciones, plazos de cancelación y no-show, fallas de verificación) obligaría a editar varios scripts, y la trazabilidad de HU11 y HU15 dependería de acordarse de registrar en cada uno.	**Descartado**
Table Module	La información del hotel es muy tabular (habitaciones, reservas, cuentas, consumos) y los reportes HU13 a HU15 trabajan sobre ese tipo de datos. Es un punto medio de esfuerzo entre los otros dos patrones.	Agrupa por tabla y no por comportamiento: no modela el flujo ni las transiciones de cada excepción (cancelación, no-show, cambio de habitación) y no relaciona el estado de la habitación con la reserva o la estancia.	Las excepciones quedarían repartidas entre varios módulos y sería difícil proteger invariantes que cruzan tablas, como que no haya check-in sin identidad y garantía.	**Descartado**
Domain Model	La Habitación, la Estancia, la Reserva y la Cuenta protegen sus reglas y estados. Disponibilidad y tarifa se definen una vez y se reutilizan. Cambio de habitación y consumos son hechos trazables y las reglas se prueban sin pantallas ni almacenamiento.	Requiere más diseño previo y puede caer en sobrediseño o en dominio anémico (objetos con datos pero sin reglas). También existe el riesgo de inventar reglas que el negocio no definió.	Más clases y más esfuerzo inicial (costo aceptado). Cada regla nueva se agrega en el objeto responsable. Exige modelar solo los conceptos del glosario y marcar las reglas pendientes (ver Riesgos y reglas pendientes).	**Elegido**

Tabla comparativa

Domain Model es el patrón que mejor encaja en 6 de los 7 criterios evaluados; Transaction Script solo lo supera en esfuerzo inicial.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Medio: cada operación es sencilla, pero el cálculo de tarifa y la disponibilidad se repiten en cada script.	Medio: se concentra en un módulo por tabla, pero sin relación con el estado de la reserva.	Alto: la tarifa y la disponibilidad son conceptos del negocio y cada objeto acepta o rechaza la operación según su estado.	BRD (límite de 120 habitaciones), HU02, HU14: tarifa según temporada, tipo de habitación y condiciones comerciales.
Cambios esperados	Bajo: cada nueva regla obliga a tocar varios scripts.	Medio: la regla se agrega en un módulo, pero si cruza tablas sigue repartida.	Alto: la regla se agrega en el objeto responsable (Reserva, Estancia o Servicio de tarificación).	Entrevista y vacíos del PDF: penalizaciones, plazos de cancelación y no-show, fallas de verificación y condición comercial sin definir.
Interacción entre conceptos	Bajo: la disponibilidad se duplica o se extrae a funciones sueltas.	Medio: se reutiliza dentro del módulo.	Alto: una sola regla de disponibilidad reutilizada por todos los casos.	HU01, HU02, HU07, HU09, HU11: la disponibilidad se usa en reserva, cambio de habitación, extensión, cancelación y no-show.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Estados e invariantes	Bajo: cada script revisa y cambia estados por su cuenta y crecen los condicionales anidados.	Medio: un módulo de habitaciones centraliza el campo estado, pero no protege invariantes que cruzan tablas.	Alto: la Habitación controla sus transiciones válidas, la Estancia no inicia sin identidad y garantía, y cada objeto rechaza operaciones inválidas.	Glosario (Disponible, Ocupada, Sucia, Limpia, Fuera de Servicio), HU03, HU06, HU10 y las 7 excepciones del BPMN.
Criterio adicional: trazabilidad	Bajo: depende de acordarse de registrar en cada script.	Medio: se registra por tabla.	Alto: el cambio de habitación y los consumos son hechos del negocio con usuario y hora.	Entrevista, HU11, HU15: 100% de trazabilidad en cambios de habitación y consumos.
Criterio adicional: facilidad de prueba de reglas	Medio: se prueba por script completo.	Medio	Alto: las reglas se prueban sin pantallas ni almacenamiento.	Criterio de éxito de la gerencia: disponibilidad sincronizada y libre de errores humanos (BRD).
Criterio adicional: esfuerzo de implementación inicial	Alto ajuste: es el más rápido de construir.	Medio	Medio: requiere diseño previo del dominio.	Alcance del BRD: 15 historias y glosario de 20 términos.
Ajuste global	**Bajo a medio**	**Medio-bajo**	**Alto**	Domain Model supera a los otros en 6 de 7 criterios.

Decisión argumentada

Se elige **Domain Model** como patrón principal de la lógica de negocio de todo el proceso (reserva, check-in, estancia, check-out y limpieza), acompañado de una capa de servicios de aplicación (documento 03) que solo coordina y no contiene reglas.

No se propone una combinación de patrones. Los reportes y consultas (HU13 a HU15) no usan Table Module: no contienen reglas de negocio y se resuelven como consultas con vistas de lectura (CQRS, documento 03).

La decisión se apoya en seis evidencias de la Unidad 2:

1. **Reglas que se cruzan entre casos** (BRD, HU01, HU02, HU07, HU09, HU11). La disponibilidad sobre las 120 habitaciones y la tarifa (temporada, tipo de habitación, condiciones comerciales) intervienen en reserva, extensión, salida anticipada, cancelación y no-show.
2. **La habitación tiene un ciclo de estados** (glosario, HU06, HU10). El glosario define Disponible, Ocupada, Sucia, Limpia y Fuera de Servicio, y la historia HU10 exige evitar asignaciones erróneas.
3. **Siete excepciones con caminos distintos** (BPMN). Cancelación, no-show, salida anticipada, extensión, habitación fuera de servicio, cambio de habitación y llegada anticipada.
4. **La trazabilidad es un criterio de éxito** (entrevista, HU11, HU15). La gerencia pidió 100% de trazabilidad en cambios de habitación y consumos, con motivo, hora y usuario.
5. **Las reglas todavía no están cerradas** (entrevista y vacíos del PDF). Penalizaciones, plazos de cancelación y no-show y fallas de verificación están pendientes. Con Domain Model cada regla nueva se agrega en el objeto responsable, sin tocar varios scripts.
6. **El negocio ya tiene su lenguaje** (glosario). Los 20 términos del glosario se traducen directo en objetos del dominio (Reserva, Estancia, Habitación, Cuenta, Consumo Adicional).

Qué problema concreto ayuda a resolver. Domain Model resuelve tres problemas del hotel: (1) evita escribir de nuevo la disponibilidad y la tarifa en reserva, extensión y salida anticipada; (2) hace que la Habitación y la Estancia rechacen operaciones inválidas, como asignar una habitación Fuera de Servicio o iniciar una estancia sin identidad y garantía; y (3) convierte la liberación y la limpieza de la habitación en hechos del negocio (Habitación liberada, Habitación limpiada), lo que elimina el desfase de 15 a 30 minutos que hoy existe entre limpieza y recepción.

Qué costo introduce. Requiere diseñar el dominio antes de programar y genera más clases que un script por operación. El equipo acepta ese esfuerzo inicial como la consecuencia negativa principal de la decisión. El sobrediseño se controla modelando solo los conceptos del glosario y de las 15 historias, y el dominio anémico se evita dejando las reglas en las entidades y no en los servicios.

Por qué se descartan los otros dos. Transaction Script se descarta porque los pasos de notificación y registro quedarían como tareas manuales dentro de cada script, que es justo la falla actual del hotel. Table Module se descarta porque agrupa por tabla y no modela el flujo ni las transiciones de cada excepción.

Riesgos y reglas pendientes

El principal riesgo de Domain Model es el sobrediseño, y se controla modelando solo los conceptos que el glosario del negocio ya usa.

Riesgo	Mitigación
Sobrediseño: más clases y más esfuerzo inicial	Modelar solo los conceptos del glosario y de las 15 HU
Dominio anémico: objetos con datos pero sin reglas	Las reglas viven en las entidades y los servicios de aplicación solo orquestan
Inventar reglas que el negocio no definió	Las reglas pendientes quedan marcadas y se validan con el stakeholder antes de implementarse

Reglas pendientes de validar con el stakeholder (no se inventaron para acomodar la arquitectura):

- Penalizaciones por cancelación y por salida anticipada.
- Plazos de cancelación y criterio para declarar no-show.
- Qué ocurre cuando falla la verificación de identidad o de garantía en el check-in.
- Condiciones de la llegada anticipada (early check-in) y detalle de la condición comercial.
- Que la estancia no pueda cerrarse con cargos sin consolidar o saldo sin liquidar (regla derivada del glosario).

Anexo. Evidencia por caso de uso

Tres de los cinco casos (CU-02, CU-04 y CU-05) solo se resuelven limpiamente con Domain Model; CU-01 y CU-03 podrían vivir como Transaction Script si se tomaran aislados, pero comparten reglas con los demás.

Caso	Reglas que intervienen	Qué pasa con Transaction Script	Qué pasa con Domain Model
CU-01 Confirmar reserva	Límite de 120 habitaciones, tarifa por temporada, tipo y condición comercial, rechazo si no hay disponibilidad	Un script lo resuelve, pero la disponibilidad y la tarifa se vuelven a escribir en extensión (HU09) y salida anticipada (HU08)	Disponibilidad y Tarifa son conceptos reutilizados por todos los casos que cambian fechas
CU-02 Realizar check-in	Identidad y garantía obligatorias, solo se asigna una habitación disponible, llegada anticipada depende del estado de la habitación	Condicionales anidados que revisan varios estados a la vez	La Estancia solo inicia si identidad y garantía son válidas, y la Habitación solo se asigna si está disponible
CU-03 Registrar consumo adicional	Solo recepción, restaurante y lavandería pueden cargar; el cargo va a la cuenta de la estancia	Es el caso más simple: una lista de roles y un registro	La Cuenta rechaza cargos de roles no autorizados y se mantiene consolidada
CU-04 Cambiar de habitación	Destino disponible y no fuera de servicio; historial con motivo, hora, usuario y habitaciones	El historial depende de que cada script lo guarde	La Estancia registra el cambio como un hecho trazable y la Habitación destino valida su propio estado
CU-05 Hacer check-out y liberar	Cargos consolidados, saldo liquidado, habitación pasa a limpieza y regresa a disponible, recepción debe verla de inmediato	Notificar a limpieza y a recepción queda como pasos manuales dentro del script, justo la falla actual	La liberación de la Habitación produce un hecho del negocio al que reaccionan limpieza y recepción